Retail Data Lake

Analyse des Ventes Retail en Tunisie

Master FAVD — Fouille, Analyse et Visualisation des Données

Étudiante	Safa Jlassi & Ines Boujemaa
Filière	Master FAVD
Module	Visualisation des Données Massives
Année	2025 – 2026
Date	01/05/2026

Université : Faculté des Sciences Juridiques, Économiques et de Gestion de Jendouba

1. Introduction

Ce rapport présente la conception et l'implémentation d'un Data Lake complet dédié à l'analyse des ventes retail en Tunisie. Le projet s'inscrit dans le cadre du module **Visualisation des Données Massives** du Master FAVD.

L'objectif principal est de simuler un environnement Big Data réaliste en générant 100 000 transactions commerciales fictives, de les stocker et traiter via une stack distribuée (HDFS + Spark), puis de produire des indicateurs analytiques exploitables à travers un dashboard interactif.

Problématique

Comment concevoir un pipeline de données end-to-end capable d'ingérer, stocker, traiter et visualiser de grands volumes de données transactionnelles dans un contexte retail tunisien ?

Objectifs

- Générer un jeu de données réaliste représentant le marché retail tunisien
- Mettre en place une architecture Data Lake avec HDFS comme couche de stockage distribué
- Traiter les données avec Apache Spark pour calculer des KPIs métier
- Visualiser les résultats via Jupyter Notebook et un dashboard Streamlit interactif
- Versionner et publier le projet sur GitHub

2. Architecture proposée

L'architecture du projet repose sur une stack Big Data conteneurisée orchestrée par Docker Compose. Chaque composant est isolé dans son propre container et communique via un réseau Docker interne.

Composants principaux

Composant	Rôle	Port
Docker Compose	Orchestration de tous les services	—
HDFS (Namenode + Datanode)	Stockage distribué des données brutes	9870
Apache Spark	Traitement batch et calcul des KPIs	8080
Jupyter Notebook	Analyse exploratoire et visualisations	8888
Streamlit	Dashboard interactif et filtres dynamiques	8501

Structure du projet

```
retail-datalake/
■■■ data/generate_data.py     # Génération des données (Faker)
■■■ docker/docker-compose.yml # Orchestration Docker
■■■ hdfs/                     # Configuration HDFS
■■■ notebooks/               # Notebook + 6 PNG
■■■ raw/ventes_retail.csv     # Données brutes (100k lignes)
■■■ spark/                   # Jobs Spark
■■■ assets/                  # Schémas et diagrammes
■■■ app.py                   # Dashboard Streamlit
```

Flux de données

Le pipeline de données suit les étapes suivantes : **Génération** (Python/Faker) → **Stockage brut** (raw/CSV) → **Ingestion HDFS** → **Traitement Spark** → **Analyse Jupyter** → **Visualisation Streamlit**.

3. Modélisation des données

Le jeu de données est structuré autour d'une table de faits principale représentant les transactions de vente, enrichie de dimensions métier contextualisées au marché tunisien.

Structure du fichier ventes_retail.csv

Colonne	Type	Description
transaction_id	String	Identifiant unique de la transaction
date	Date	Date de la transaction (2022–2024)
year / month	Integer	Année et mois extraits de la date
store_name	String	Nom du magasin
region	String	Région tunisienne (Tunis, Sfax, Sousse...)
category	String	Catégorie de produit
product_name	String	Nom du produit
quantity	Integer	Quantité vendue
unit_price	Float	Prix unitaire (TND)
discount_pct	Float	Pourcentage de remise appliqué
total_ttc	Float	Chiffre d'affaires total TTC (TND)
segment	String	Segment client (Premium, Standard, Budget)

Le dataset couvre **3 années** (2022–2024), plusieurs **régions tunisiennes**, différentes **catégories de produits** et **3 segments clients**, offrant ainsi une richesse analytique suffisante pour des analyses multidimensionnelles.

4. Implémentation

4.1 Génération des données

Le script **data/generate_data.py** utilise la bibliothèque **Faker** pour générer 100 000 transactions fictives réalistes. Les données sont adaptées au contexte tunisien : noms de magasins, régions, et produits représentatifs du marché local.

```
python data/generate_data.py
# Génère raw/ventes_retail.csv avec 100 000 lignes
```

4.2 Infrastructure Docker

L'ensemble des services est orchestré via Docker Compose. Une seule commande suffit pour démarrer l'intégralité de l'environnement :

```
docker-compose -f docker/docker-compose.yml up -d
```

Les services démarrés : Namenode HDFS, Datanode HDFS, Spark Master, Jupyter Notebook.

4.3 Ingestion dans HDFS

Les données brutes sont ingérées dans HDFS via les commandes Hadoop standard, simulant un environnement de stockage distribué réel. Le fichier CSV est placé dans un répertoire dédié sur HDFS.

4.4 Traitement avec Apache Spark

Les jobs Spark lisent les données depuis HDFS et calculent les agrégations nécessaires à la production des KPIs. Spark est choisi pour sa capacité à traiter de gros volumes de données de façon distribuée et performante.

4.5 Dashboard Streamlit

Le fichier **app.py** implémente un dashboard interactif accessible sur **localhost:8501**. Il charge les données depuis raw/ventes_retail.csv et propose des filtres dynamiques par région, année et catégorie.

```
streamlit run app.py --server.port 8501
```

5. Tests

Des vérifications ont été effectuées à chaque étape du pipeline pour s'assurer de la cohérence et de l'intégrité des données.

Test	Résultat
Volume des données générées	✓ 100 000 lignes confirmées
Absence de valeurs nulles dans les colonnes clés	✓ Aucune valeur manquante
Cohérence des dates (2022–2024)	✓ Plage temporelle correcte
Calcul du total_ttc	✓ unit_price × quantity × (1 - discount_pct)
Chargement dans HDFS	✓ Fichier accessible depuis Spark
Affichage Streamlit avec filtres	✓ Mise à jour dynamique fonctionnelle
Compatibilité des chemins Windows	✓ Path relatif via pathlib

6. Résultats analytiques

Six visualisations ont été produites à partir des données traitées, couvrant les principaux axes d'analyse métier :

#	KPI	Visualisation	Insight principal
1	CA par Région	Graphique en barres	Identification des régions les plus performantes
2	Évolution mensuelle	Courbe temporelle	Tendances et saisonnalité 2022–2024
3	CA par Catégorie	Barres horizontales	Catégories générant le plus de revenus
4	Impact Promotions	Barres comparatives	Effet des remises sur le CA total
5	Top 10 Magasins	Barres horizontales	Classement des magasins les plus performants
6	Segments Clients	Camembert	Répartition du CA par profil client

Ces visualisations sont disponibles en format PNG dans le dossier **notebooks/** et sont accessibles de façon interactive via le dashboard Streamlit avec filtrage dynamique par région, année et catégorie de produit.

7. Discussion

Points forts du projet

- Architecture complète et reproductible grâce à Docker Compose
- Pipeline end-to-end de la génération à la visualisation
- Dashboard interactif avec filtres dynamiques en temps réel
- Données contextualisées au marché tunisien (régions, produits, segments)
- Code versionné et publié sur GitHub pour la reproductibilité

Limites

- Les données sont simulées et ne reflètent pas de vraies transactions commerciales
- Le volume (100k lignes) reste modeste pour un vrai environnement Big Data

Perspectives d'amélioration

- Intégration de vraies données open data tunisiennes
- Mise en place d'un pipeline temps réel avec Apache Kafka
- Séparation des environnements DEV / TEST / PROD
- Ajout d'un module de Machine Learning pour la prédiction des ventes
- Déploiement cloud (AWS S3 + EMR ou Azure HDInsight)

8. Conclusion

Ce projet a permis de concevoir et implémenter un Data Lake complet simulant un environnement Big Data retail tunisien. En combinant **HDFS** pour le stockage distribué, **Apache Spark** pour le traitement, et **Streamlit** pour la visualisation interactive, nous avons produit un pipeline end-to-end fonctionnel et reproductible.

Les six KPIs analytiques calculés offrent une vue complète des performances commerciales : par région, par catégorie, par magasin et par segment client, sur une période de trois années (2022–2024).

Ce projet constitue une base solide, extensible vers des cas d'usage plus avancés tels que l'analyse en temps réel, le Machine Learning prédictif ou le déploiement sur infrastructure cloud.

Projet disponible sur GitHub : **github.com/safajl/retail-datalake**